

LiquidHaskell: Experience with Refinement Types in the Real World

Niki Vazou, Eric L. Seidel, Ranjit Jhala

Szabó Márk

Haskell is a **statically typed** language.

- Every expression has a type.
- Types are checked before the program runs.
- Types are often inferred automatically.

Basic Types

Some common built-in types:

```
1 Int      -- integer
2 Float    -- floating point number
3 Double   -- floating point number
4 Num      -- [Int, Float, Double, ...]
5 Bool     -- True or False
6 Char     -- single character
7 String   -- text
```

Type Annotations

The operator `::` means “has type”.

```
1 age :: Int
2 age = 23
3
4 name :: String
5 name = "Mark"
6
7 active :: Bool
8 active = True
```

Function Types

The arrow $\rightarrow$ separates input and output types.

```
1 add :: Int -> Int -> Int
2 add x y = x + y
```

- First Int: first argument
- Second Int: second argument
- Last Int: result

Equivalent in a C-like language:

```
1 int add(int x, int y) { return a + b; }
```

Type Inference

Haskell can infer many types automatically.

```
1 age = 23
2 double x = x * 2
```

In GHCi we can inspect inferred types:

```
1 :t True
2 True :: Bool
3
4 :t "hello"
5 "hello" :: String
6
7 :t double
8 double :: Num a => a -> a
```

Limits of The Type System

We can't define constraints for the types of a function like:

- Conditions about the value: $x > 0$
- Relationships between values: $a > b$

To check for positive we need to check inside the function:

```
1 sqrtRoot :: Double -> Double
2 sqrtRoot x
3   | x > 0 = sqrt x
4   | otherwise = error "x must be greater than 0"
```

Many of these properties and relationships can be verified using a combination of **Refinement Types** and **external SMT solvers**.

This paper introduces and experiments with **LiquidHaskell** a refinement typechecker used to verify more than **10.000** lines of Haskell code across popular libraries.

Including: *bytestring*, *text*, *vector-algorithms* and *xmonad*.

Refinement Types